

Git and GitHub

Keep your project history, work with your team, and show what you have built.

Arif Solmaz
İSTÜN Mechatronics Engineering



The 45-minute route

One robot project, four stops.

1. Basics: files, repositories, and history · 9 min 15 sec
2. Changes: from your computer to GitHub · 12 min 45 sec
3. Teamwork: issues, branches, and review · 12 min
4. Sharing, resources, and questions · 11 min

32 main slides · 12 reference slides · One project throughout

Which file is the latest?

Is robot_final_SON.ino really the final version?

THE PROBLEM

Which one is the latest?

23:47

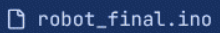
Robot project


robot.ino


history: every change, who, when

Robot team

Kaan
Elif, can you send the latest code?

Elif
Sent robot_final.ino


Deniz
But I fixed the sensor in robot_v2...

Kaan
The robot left the line again. Which one is the latest??

Elif
No idea. Should we try them one by one?

On GitHub there is one `robot.ino` ; its old versions live in its history.

We keep a history of changes instead of putting version names into filenames.

Git and GitHub

Git keeps history. GitHub supports collaboration around that history.

Git • the tool on your computer

Manages commits, branches, and local history.
Many operations work without an internet connection.

Git also works with GitLab and other Git servers.

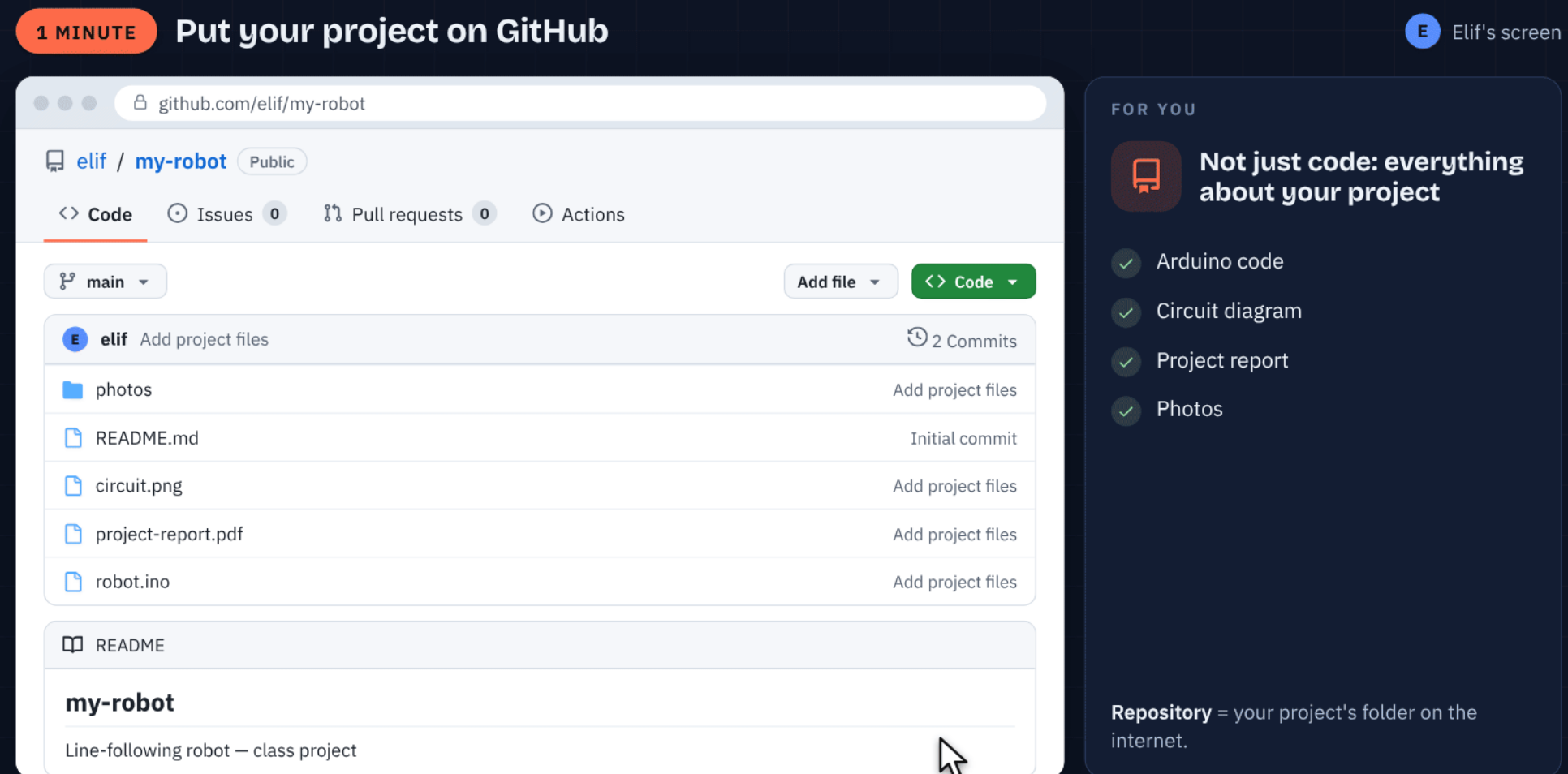
GitHub • the shared platform

Hosts repositories and brings issues, pull requests, and review tools together.

01 · BASICS

A repository holds files and history

Code, circuit diagrams, reports, and documentation belong to the same project.



Done: code, circuit diagram, report and photos are now online.

Repository · robot.ino + README.md + circuit diagram + recorded history

A commit is a checkpoint

“Increase the IR sensor threshold” describes a change and its purpose.

- Records a selected state of the project in its history.
- Has an ID, an author, and a descriptive message.
- Committing locally is separate from sending changes to GitHub.

Save ≠ commit ≠ push

The README is the entry point

A visitor should understand the project in a few sentences.

```
# My Robot Project
```

```
An Arduino robot that follows a line using IR sensors.
```

```
## Hardware
```

```
Arduino, IR sensors, motor driver, and motors.
```

```
## Running the project
```

```
Follow the wiring diagram and the comments in robot.ino.
```

Purpose • hardware • setup • photograph or diagram

Who can see it?

Visibility and permission to write are separate decisions.

Public

Anyone can view and clone it. Writing to the original repository requires permission.

Private

Access is limited to authorized people. Team members receive suitable permissions.

Passwords, access keys, and personal data do not belong in project files.

02 · THE PATH OF A CHANGE

It worked yesterday

History gives a concrete answer to “What changed?”

2 BENEFIT

A time machine for your project

E Elif's screen

FOR YOU

A time machine for your project

✓ No change is ever lost.

✓ Who changed what, and when: all recorded.

✓ Don't be afraid to break things: every save is a way back.

Commit = a save point with a short note.

github.com/elif/my-robot/commit/7f3e2a1

elif / my-robot Public

<> Code Issues 0 Pull requests 0 Actions

Change sensor threshold

deniz committed yesterday 7f3e2a1

<> Browse files

1 file changed

robot.ino +1 -1

3 int TURN_SPEED = 120; // speed in turns

4 - int THRESHOLD = 500; // sensor threshold

4 + int THRESHOLD = 900; // sensor threshold

5

6 void setup() {

Found it: the threshold went from 500 to 900. You can look at the project as it was that day.

Who, when, and in which commit? · History contains committed changes.

How to read a diff

A diff shows the changes between two states.

```
robot.ino
```

```
- const int ESIK = 500;  
+ const int ESIK = 900;
```

– earlier line · + new line · ESIK: example threshold for an IR sensor reading

A change passes through four areas

Edit → stage → record → share



Add stages changes. Commit records them locally. Push sends them. Pull brings remote updates in.

Clone creates a local copy

Bring an existing repository to your computer, with its files and history.

```
git clone <repository-url>  
cd robot-projem
```

Public repositories can be read over HTTPS; private ones require access. A ZIP has no Git history.

First, edit the file

We added the line sensors' purpose to the README and saved the file.

```
README.md
```

```
## Sensors
```

```
IR sensors detect the line.
```

```
The wiring is shown in devre-semasi.png.
```

```
# Save the file → then:
```

```
git status
```

Without an edit, add and commit do not create a new change.

Add selects the change

Choose the scope of a commit deliberately.

```
git add README.md  
git diff --cached
```

Add stages the file's current state. Later edits may need another add.

Commit records the change locally

The message explains what the change accomplishes.

```
git commit -m "Document IR sensor wiring in README"  
git log --oneline
```

This commit is on your computer. Sending it to GitHub is the next step.

Push sends commits to GitHub

Local commits become visible remotely after a successful push.

```
git push
```

Write permission and authentication are required. Your GitHub password is not a Git HTTPS password.

Pull brings updates in

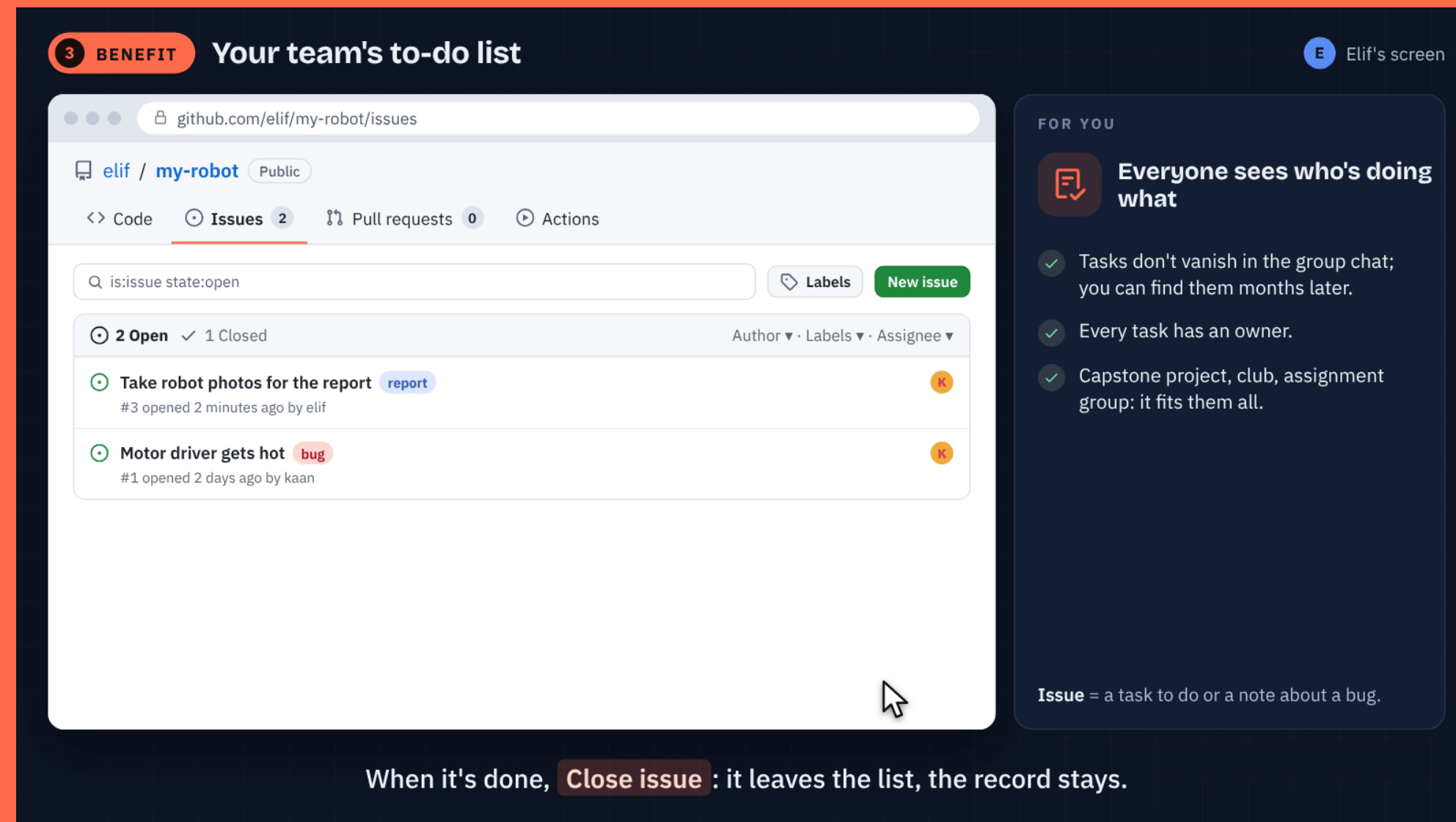
The shared history and the local copy come together again.

```
git status  
git pull --ff-only
```

This example proceeds if the local branch has no separate commits. Diverged histories need a chosen integration method.

An issue makes work visible

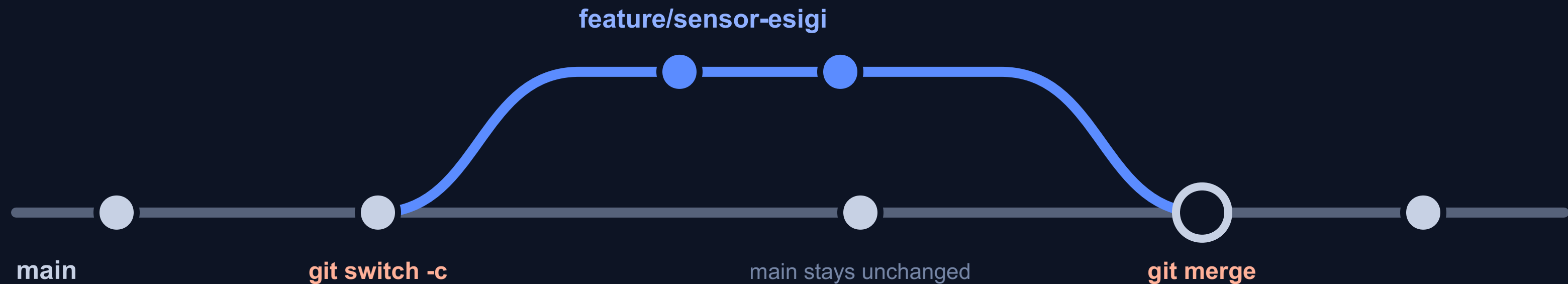
“Investigate the IR sensor threshold” becomes a shared work item.



Title + observed behavior + expected outcome + owner

A branch is a separate line of work

Different changes can grow from the same starting point.



A branch is a name pointing to a commit, rather than a physical copy of every file.

Start with a simple branch workflow

The project's needs determine its branch workflow.

The main line

main • the team's accepted current work.

Task branches

feature/sensor-esigi • a specific change, review, and merge.

dev and staging are not required for every project. Read CONTRIBUTING to find the target branch.

A pull request is a proposal

“Shall we bring this branch’s change into main?”

Source: feature/sensor-esigi → Target: main

Title: Increase the IR sensor threshold

Why: The robot was losing the line in some conditions.

Change: ESIK, 500 → 900.

Validation: State the observations and tests performed here.

A PR explains what changed, why, and how it was checked.

Review strengthens the solution

A small number change can have a large effect on the robot's behavior.

- Files changed: modified lines and their context.
- Comment: “Under what conditions was this threshold tested?”
- Pushing a new commit to the same branch updates the PR.
- Merge after approval and any required checks.

Review keeps the decision and its reasoning with the project.

Update locally after the merge

A change accepted on GitHub must also be brought into local main.

```
git switch main  
git pull --ff-only
```

1. Merge the PR into main.
2. Switch to the local main branch.
3. Bring in remote updates.
4. Remove the merged task branch if no longer needed.

Before deleting a branch, check that its change is present in the target branch.

A conflict asks us to decide

Two branches changed the same line in different ways.

```
robot.ino
```

```
<<<<<< HEAD
```

```
const int ESIK = 900;
```

```
=====
```

```
const int ESIK = 700;
```

```
>>>>>> main
```

HEAD: current branch · main: incoming branch · ESIK: IR sensor reading threshold

Resolve the meaning of the conflict

Decide on the correct behavior, then remove the markers.

```
git add robot.ino  
git commit -m "Resolve the IR sensor  
threshold conflict"
```

1. Read the reasons and context behind both changes.
2. Choose a suitable solution or write a new one.
3. Remove markers, save the file, and validate the result.
4. Stage the file with add and complete the merge commit.

This is a merge example; the command to continue depends on the operation.

How do we choose existing code?

When adding an HC-SR04 distance sensor, the first search result is not enough.

- Compatibility with the hardware and software version.
- README, wiring information, and working examples.
- Maintenance, open issues, and releases.
- License and conditions of use.

A star signals interest or a saved repository; it does not guarantee quality or usage.

A license sets the conditions of use

Being able to see code does not mean being allowed to use it for every purpose.

- Read LICENSE and the licenses of dependencies.
- Check conditions for copying, distribution, and modification.
- Keep required attribution and license notices.

For open-source code, consider both technical suitability and permission to use it.

Contribute through a fork

Work in your own copy and propose a change to the original project.

1. Read the contribution guide and find the target branch.
2. Fork the repository to your account, then clone your fork.
3. Edit on a task branch, commit, and push to your fork.
4. Open a PR against the original project's correct target branch.

You can clone a public repository directly. A fork gives you your own remote copy to work in.

A portfolio explains your work

A few clearly documented projects can say more than a contribution count.

Show in the project

Purpose, your contribution, hardware, results, and operating limits.

Organize on your profile

Featured repositories, clear READMEs, and working project links.

The contribution graph shows activity GitHub counts, rather than every workday or all your effort.

Student resources

GitHub Education offers tools and learning opportunities for verified students.

- Student Developer Pack: changing partner offers.
- Copilot Student: free access for verified students.
- Application: verify student status with the required evidence.

Current terms and offers: education.github.com/pack

RECAP

The whole project workflow

Problem → issue → branch → change → commit → push → PR → review → merge → pull

- Git records history; GitHub organizes collaboration.
- Saving, committing, and pushing are separate steps.
- README explains the project, an issue the task, and a PR the solution.

Resources: [GitHub Docs](#) · [Pro Git](#) · [GitHub Skills](#)

CLOSING

Questions

Where is the change? Who can see it? How does it reach the team?

Git · GitHub · repository · commit · branch · issue · pull request

[GitHub Docs](#)

[Pro Git · English](#)

[GitHub Skills](#)

[GitHub Education](#)

Installing Git

Use the official installation route for your operating system.

```
git --version
```

Windows

Git for Windows, with Git Bash or a suitable terminal.

macOS and Linux

A package or installation method suited to the operating system.

Version numbers vary. Current instructions: git-scm.com/install

Commit identity

Configure the name and email that will appear in commits.

```
git config --global user.name "Elif Örnek"  
git config --global user.email "elif@example.com"  
git config --global init.defaultBranch main
```

The name and email are examples. These settings do not authenticate you to GitHub.

Editor and terminal

The same Git operation can be done in a terminal or a graphical interface.

Terminal

Commands and options are explicit, making steps easy to share.

VS Code

Files, diffs, Source Control, and an integrated terminal in one window.

Open the project with Open Folder; show the terminal with View → Terminal.

.git and .gitignore

The history database and tracking preferences are stored separately.

```
robot-projem/  
  .git/  
  .gitignore  
  README.md  
  robot.ino  
  
# Example .gitignore  
build/  
.env
```

Do not edit .git by hand. .gitignore does not remove tracked files from history.

An SSH key pair

Add the public key to GitHub; keep the private key to yourself.

```
ssh-keygen -t ed25519 -C "elif@example.com"
```

```
# File to add to GitHub:
```

```
~/.ssh/id_ed25519.pub
```

```
# Private key: do not share
```

```
~/.ssh/id_ed25519
```

A passphrase adds protection. Do not overwrite an existing key.

SSH agent and connection

Agent setup depends on the operating system.

```
# Bash / Git Bash example
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# Check the GitHub connection
ssh -T git@github.com
```

macOS Keychain and Windows OpenSSH steps may differ; follow the official instructions.

Turn a local folder into a repository

This example assumes an empty remote repository on GitHub.

```
cd robot-projem  
git init -b main  
git add README.md robot.ino  
git commit -m "Initial robot project commit"  
git remote add origin <repository-url>  
git remote -v
```

README.md and robot.ino already exist. If the remote has initial commits, clone may be simpler.

First push and tracking

-u records which remote branch the local branch tracks.

```
git push -u origin main
```

```
# After setting up tracking:
```

```
git push
```

```
git pull --ff-only
```

Check the branch name with `git branch`. Commands must use the project's actual branch name.

Branch and diff commands

Make the before and after sides of a comparison explicit.

```
git branch
git switch -c feature/sensor-esigi

# Compare branch tips: main → feature
git diff main feature/sensor-esigi -- robot.ino

# Feature changes from the common starting point:
git diff main...feature/sensor-esigi
```

Tab completion depends on the shell. The typed prefix must match the start of the branch name.

Choose what you want to undo

Files, the staging area, and history require different operations.

Unstage a change

`git restore --staged README.md` The edit stays in the working file.

Reverse a shared change

`git revert <commit>` Creates a new commit that reverses an earlier effect.

`git reset HEAD~1` moves the branch to the previous commit; the default mixed mode preserves working files.

Hard reset and recovery limits

A hard reset can discard uncommitted work.

```
# Only after checking the effects:  
git reset --hard <commit>  
  
# Inspect local reference movements:  
git reflog
```

Reflog may help find earlier commits; it cannot guarantee recovery of all unrecorded work.

Quick command reference

Commands are shorthand for the concepts in the main lecture.

```
git clone <repository-url>
# Edit and save the file
git status
git add <file>
git commit -m "What changed and why?"
git push

# Get the team's updates
git pull --ff-only
# Create a task branch
git switch -c feature/<task>
```

Replace placeholders with real values. Installation, identity, and permissions must be in place.